Project 2 - Laser Tag Simulation*

Alex Dhrubo Malakar
School of Electrical & Computer Engineering
Oklahoma State University
Stillwater, Oklahoma, U.S.A
dhrubo.malakar@okstate.edu

***Abstract*—A centralized infrared laser tag system was designed and fully simulated in Python using object-oriented programming and multithreading. The prototype implements realistic player state management, hit validation with probabilistic accuracy, non-blocking reload and respawn mechanics, centralized game control, and scientific visualization of match results. The architecture is directly translatable to embedded hardware while providing complete observability during development.**

***Index Terms*—**

I. System Overview

The proposed laser tag system is a centralized, infrared-based multiplayer arena game consisting of three primary components: player units, sensors, and a base station. Each player unit integrates an IR transmitter for firing, IR reciever for hit detection, and a microcontroller for local processing. The base station, implemented as the `GameController` class in the prototype, serves as the single authoritative source for game state, scoring, and timing.

Communication is simulated using 38 kHz modulated infrared pulses; in physical deployment this will employ IR LEDs and photo-diode receivers. The simulation preserves realistic behavior through a 70% hit success probability to model a bit of chance.

Game rules are defined as follows: each player starts with 100 health points and 30 rounds of ammunition. A successful hit inflicts 20 damage. Upon health depletion, the player is eliminated and automatically respawns after 5 seconds with full health. Ammunition is replenished via a 2-second reload action. Scoring awards 100 points per elimination. Matches run for a fixed duration of 10 minutes, after which final statistics and a score-progression graph are displayed.

II. Methods & Logic Design

Player initialization occurs through the `Player` class constructor, which assigns a unique `player_id` and team affiliation ("Red" or "Blue") to support future team modes and friendly-fire detection. Initial attributes include maximum health (100), full ammunition (30), alive status, and zero score.

The system employs a fully centralized control architecture via the `GameController` class, which maintains references to all players and manages the master clock. Time synchronization is achieved by capturing a single `start_time` at match initiation using `time.time()`; all subsequent timestamps are relative offsets, eliminating drift within the simulation and providing a straightforward path to NTP synchronization in distributed hardware.

Hit validation incorporates multiple integrity checks: the shooter must be alive and possess ammunition (decremented immediately upon firing). A 70% success probability simulates real-world infrared signal degradation. Valid hits are packaged as dictionaries containing direct references to source and target `Player` objects, preventing ID spoofing or replay attacks.

Ammunition, reload, and respawn mechanisms execute non-blockingly using Python threading. When ammunition is depleted, a daemon thread performs a 2-second reload sequence. Upon death, a separate thread handles a 5-second respawn delay before restoring full health and alive status, ensuring the main game loop remains responsive under high event rates.

Game state management is controlled centrally through a `game_active` flag and fixed-duration timer. Safety and calibration are addressed via the probabilistic hit model (representing effective signal range) and daemon threads that terminate cleanly on program exit, preventing resource leaks. This design yields a robust, real-time-capable architecture that translates directly from simulation to embedded implementation.